

Contents

At a glance

Usage

Examples

How the agent is invoked

Preflight checks

The two modes

Milestone mode (default)

Bug-fix discovery mode (--milestone 1337)

What the agent must read first

The loop and its termination

Artifacts

Interaction with auto-develop.sh

auto-create-issues.sh

Backlog generator. The script points an AI agent at the project's milestone documents and lets it file GitHub issues — one per iteration — until the agent reports that the milestone is fully covered.

It is the front half of the automation pair: `auto-create-issues.sh` fills the backlog, `auto-develop.sh` empties it.

At a glance

Property	Value
Location	<code>auto-create-issues.sh</code> (repository root)
Agents supported	<code>claude</code> , <code>codex</code> , <code>opencode</code>
Required tooling	<code>git</code> , <code>gh</code> (authenticated), and the CLI of the agent you select
Source of truth	<code>docs/MILESTONES.md</code>
Termination	Agent outputs <code>DONE</code> three times in a row
Artifacts	<code>logs/issue-creation/m<milestone>/</code>

Usage

```
./auto-create-issues.sh --agent <claude|codex|opencode> --milestone <number>
```

Both flags are mandatory; without them the script prints usage and exits with status 1.

Flag	Meaning
<code>--agent <agent></code>	Which agent runner creates the issues
<code>--milestone <n></code>	Milestone number from <code>docs/MILESTONES.md</code> (<code>0</code> , <code>1</code> , <code>2</code> , ...). The special value <code>1337</code> switches to bug-fix discovery mode
<code>-h</code> , <code>--help</code>	Print usage and exit

Examples

```
# Plan out milestone 0 with opencode
./auto-create-issues.sh --agent opencode --milestone 0

# Hunt for bugs instead of planning a milestone
./auto-create-issues.sh --agent opencode --milestone 1337

# Milestone 1 with Claude
./auto-create-issues.sh --agent claude --milestone 1
```

How the agent is invoked

Agent	Command
claude	<code>claude --dangerously-skip-permissions -p "<prompt>"</code>
codex	<code>codex exec --yolo "<prompt>"</code>
opencode	<code>opencode run "<prompt>"</code>

Permission prompts are disabled in all three cases. The agent creates issues on GitHub by running `gh` itself — the script never calls `gh issue create`.

Preflight checks

1. The agent name is one of `claude`, `codex`, `opencode`.
2. `git`, `gh`, and the selected agent's CLI are on `PATH`.
3. The current directory is inside a git repository (the script then `cd`s to the root).
4. `gh auth status` succeeds.
5. The milestone is numeric.
6. Unless the milestone is `1337`: `docs/MILESTONES.md` exists **and** contains a heading `### M<n>:`. If it does not, the script aborts. The matched heading line is passed into the agent prompt as the milestone reference.

Note that, unlike `auto-develop.sh`, this script has **no repository-owner check** — it will happily create issues in whatever repository you happen to be standing in.

The two modes

Milestone mode (default)

The agent is told to create **exactly one** next logical issue for milestone `M<n>`.

- **Title format:** `M<n>-X: <concise title>`, where the agent determines `X` as the next sequence number for that milestone.
- The GitHub milestone field **is** set on the issue.
- **Deduplication:** all open issues globally, plus closed issues of this same milestone (so solved work is not re-created).

Bug-fix discovery mode (`--milestone 1337`)

`1337` is not a real milestone. It flips the prompt into bug-hunting: the agent looks for **exactly one real bug** in the codebase and files an issue for it.

- **Title format:** `M1337-X: <concise bug title>`.
- The GitHub milestone field is **not** set (there is no such milestone).
- **Deduplication:** open issues only.
- `docs/MILESTONES.md` is not consulted.

What the agent must read first

Both prompts require the agent to read the project's intent documents before deciding what to file: `SOUL.md`, `MEMORY.md`, `AGENTS.md`, `docs/BRD.md`, `docs/story.md`, `docs/DD/*`, the relevant `docs/BRD/*` files, plus the existing issues via `gh`. In milestone mode `docs/MILESTONES.md` is added to that list.

The agent is also told to look at a solved M0 issue before drafting, so new issues follow the established body structure: **Summary** → **Requirements** → **Acceptance Criteria checklist** → **Dependencies / references**.

Labels are optional. The agent may apply any existing repository label it finds useful, but none are mandatory — see [the consequence of this](#) below.

The loop and its termination

The script runs the agent over and over. Each iteration is one issue.

Termination is by **DONE streak**: when the agent has nothing left to file, it is instructed to output `DONE` as both the first and the last word of its response. The script checks the first *and* last word of the captured output; either one matching is enough to count.

- A match increments the streak and logs `DONE detected (n/3 consecutive)`.
- **Three consecutive** matches end the run.
- Any iteration without `DONE` resets the streak to zero.

The comparison is tolerant: the word is stripped of every non-alphanumeric character and uppercased before matching, so `**DONE**`, `done.` and `DONE` all count.

There is no iteration cap. The loop's only exit is the DONE streak. An agent that keeps inventing new issues, or that never emits the token, will run — and file issues, and spend budget — indefinitely. A non-zero exit from the agent is only a warning; the loop continues.

Artifacts

Each iteration's full stdout and stderr is captured to:

```
logs/issue-creation/m<milestone>/iteration-<n>-<UTC-timestamp>.txt
```

Nothing else is written. The issues themselves live on GitHub — the script prints the file path and the detected first/last word after every iteration, which is the fastest way to see why a loop is or is not terminating.

Interaction with auto-develop.sh

The two scripts are designed to be run back to back, and two details couple them:

Title prefixes are stripped downstream. `auto-develop.sh` removes the `M<n>-X:` prefix when it builds the commit message, so the titles produced here land cleanly as conventional commit descriptions.

Missing labels degrade the commit message. `auto-develop.sh` derives the commit *type* and *scope* from the issue's labels. Since this script makes labels optional, an unlabelled issue produces a commit with the default type `feat` and no scope at all. If you care about accurate commit metadata, make sure the agent actually labels what it files — or label the issues yourself before running the development pipeline.

Only owner-created issues get picked up. `auto-develop.sh` selects issues created by the repository owner. Issues filed here under a different GitHub account will sit in the backlog untouched.